

Énoncé : Le Maximum de la Fenêtre Glissante

Contexte

Vous recevez un tableau d'entiers et une taille de fenêtre fixe k . Cette fenêtre se déplace de la gauche vers la droite, un élément à la fois. À chaque position de la fenêtre, vous ne pouvez "voir" que les k nombres qui se trouvent à l'intérieur.

Le but est de trouver le **maximum** à chaque étape du déplacement de la fenêtre.

Objectif

Écrire une fonction qui calcule et renvoie la liste de ces maximums successifs.

Consignes de codage

Vous devez écrire une fonction `max_fenetre_glissante(nums, k)` en Python.

- **nums** : Une liste d'entiers (`list[int]`).
- **k** : Un entier représentant la taille de la fenêtre ($1 \leq k \leq \text{len}(\text{nums})$).
- **Valeur de retour** : Une liste d'entiers contenant le maximum de chaque fenêtre.

⚠ Attention aux pièges d'implémentation :

- **Approche Naïve** : Extraire la sous-liste à chaque étape et faire un `max()` dessus aura une complexité de $O(n \times k)$. Sur de très grands tableaux, cela provoquera un dépassement de temps (Time Limit Exceeded).
- **Optimisation avec deque** : Vous devez utiliser une `deque` (du module `collections`) pour maintenir uniquement les indices des éléments utiles de la fenêtre actuelle. L'astuce est de garder cette `deque` **strictement décroissante** en valeur (une file monotone) afin que le maximum soit toujours accessible instantanément à l'avant (index 0) en $O(1)$.

Exemples de comportement attendu

Exemple 1 : Cas classique

```
nums = [1, 3, -1, -3, 5, 3, 6, 7]
k = 3

# Fenêtre 1 : [1, 3, -1]    -> Max : 3
# Fenêtre 2 : [3, -1, -3]   -> Max : 3
# Fenêtre 3 : [-1, -3, 5]   -> Max : 5
# Fenêtre 4 : [-3, 5, 3]    -> Max : 5
# Fenêtre 5 : [5, 3, 6]     -> Max : 6
# Fenêtre 6 : [3, 6, 7]     -> Max : 7

print(max_fenetre_glissante(nums, k))
```

```
# Sortie attendue : [3, 3, 5, 5, 6, 7]
```

Prototype de la fonction

```
from collections import deque

def max_fenetre_glissante(nums: list[int], k: int) -> list[int]:
    """
    Trouve le maximum dans chaque fenêtre glissante de taille k.

    :param nums: Liste d'entiers à analyser
    :param k: Taille de la fenêtre glissante
    :return: Liste des maximums successifs
    """
    # Votre code ici
    pass
```

Tests

```
print("Démarrage des tests pour la Fenêtre Glissante (Format Deque)...")

# Test 1 : Fenêtre de taille 1 (Le résultat est identique au tableau d'origine)
print("Test 1...", max_fenetre_glissante([1, 3, -1, 5], 1))
assert max_fenetre_glissante([1, 3, -1, 5], 1) == [1, 3, -1, 5], "Échec Test 1"

# Test 2 : Fenêtre égale à la taille du tableau
print("Test 2...", max_fenetre_glissante([1, 3, -1, 5], 4))
assert max_fenetre_glissante([1, 3, -1, 5], 4) == [5], "Échec Test 2"

# Test 3 : Tableau strictement croissant
print("Test 3...", max_fenetre_glissante([1, 2, 3, 4, 5], 3))
assert max_fenetre_glissante([1, 2, 3, 4, 5], 3) == [3, 4, 5], "Échec Test 3"

# Test 4 : Tableau strictement décroissant
print("Test 4...", max_fenetre_glissante([5, 4, 3, 2, 1], 3))
assert max_fenetre_glissante([5, 4, 3, 2, 1], 3) == [5, 4, 3], "Échec Test 4"

# Test 5 : Éléments identiques
print("Test 5...", max_fenetre_glissante([2, 2, 2, 2], 2))
assert max_fenetre_glissante([2, 2, 2, 2], 2) == [2, 2, 2], "Échec Test 5"

# Test 6 : Cas classique alternatif (valeurs négatives)
print("Test 6...", max_fenetre_glissante([1, 3, -1, -3, 5, 3, 6, 7], 3))
assert max_fenetre_glissante([1, 3, -1, -3, 5, 3, 6, 7], 3) == [3, 3, 5, 5, 6, 7],
"Échec Test 6"
```

```
# Test 7 : Test de performance (Grand tableau, grande fenêtre)
# Une approche en  $O(n*k)$  ferait environ 500 000 000 d'opérations et prendrait
plusieurs secondes/minutes.
# Une approche en  $O(n)$  avec deque prend quelques millisecondes.
import random
grand_tableau = [i for i in range(50000)] # Tableau croissant pour simplifier la
validation
k_grand = 10000
print("Test 7 (Test de performance)...")
resultat_perf = max_fenetre_glissante(grand_tableau, k_grand)
# Le premier max de la fenêtre [0...9999] doit être 9999, le dernier doit être
49999
assert len(resultat_perf) == 40001, "Échec Test 7 : Taille du résultat incorrecte"
assert resultat_perf[0] == 9999 and resultat_perf[-1] == 49999, "Échec Test 7 :
Valeurs incorrectes ou trop lent"

print("\nTous les tests de la Fenêtre Glissante ont passé avec succès !")
```